

Digital Logic Design

**Semester II
FY**

**Course Code
316U06C203
Jan – May 2026**

Course Outcomes

At the end of the successful completion of the course, the student will be able to

C01: Understand the fundamentals of number systems, codes, and arithmetic operations.

C02: Represent, implement, and minimize logic functions using different techniques.

C03: Implement combinational logic circuits and their applications using basic gates and MSI devices.

C04: Implement sequential logic circuits and applications using MSI devices.

C05: Understand basic principles required for advanced Digital Design

Number Representation and Codes

Number Representation and Codes

- **Number System for Digital Systems / Computers**
- **Review of Positional Number System**
- **Binary, Octal, Decimal, Hexadecimal Numbers**
- **Conversion of Numbers : Decimal to others , others to decimal**
- **Representation of Numbers**
 - **Unsigned**
 - **Signed**
- **2's complement , sign magnitude**
- **Binary Addition**
- **Binary Subtraction: 1's complement, 2's complement**
- **Gray Code, BCD Code**
- **Parity for error détection**

Positional Number System: Concept

b = base of the number system

POSITIONAL VALUE	b^3	b^2	b^1	b^0	Actual value in decimal
Digit representation	d3	d2	d1	d0	$d0 \times b^0 + d1 \times b^1 + d2 \times b^2 + d3 \times b^3$
Value Decimal	1000	100	10	1	
Value (Binary)	8	4	2	1	
Value (Octal)	512	64	8	1	
Value (Hexadecimal)	4096	256	16	1	

Positional Number System : Decimal Numbers

POSITIONAL VALUE	b^3	b^2	b^1	b^0	Actual value in decimal
Representation Decimal	4	2	7	1	
Positional Value	1000	100	10	1	
Value	4000	200	70	1	4271
Representation Decimal	6	5	4	2	
Positional Value	1000	100	10	1	
Value	6000	500	40	2	6542

Positional Number System : Binary, Octal, Hexadecimal

POSITIONAL VALUE	b^3	b^2	b^1	b^0	Actual value in decimal
Representation binary	1	0	1	1	
Value (Binary)	8	0	2	1	11
Representation Octal	4	3	1	0	
Value (Octal)	512	64	8	1	2248
Representation decimal	1	0	2	1	
Value Decimal	1000	100	10	1	1021
Representation hex	2	A	5	B	
Value (Hexadecimal)	4096	256	16	1	10843

Conversion from Decimal to Any number system

Base	Given number	Remainder
2	19	
2	9	1
2	4	1
2	2	0
2	1	0
	0	1

$$19 = (10011)_2$$

Base	Given number	Remainder
8	19	
8	2	3
	0	2

$$19 = (23)_8$$

Base	Given number	Remainder
16	19	
16	1	3
	0	1

$$19 = (13)_{16}$$

Conversion from Decimal to Any number system

Base	Given number	Remainder
2	75	
2	37	1
2	18	1
2	9	0
2	4	1
2	2	0
2	1	0
	0	1

$$75 = (1001011)_2$$

Base	Given number	Remainder
8	75	
8	9	3
8	1	1
	0	1

$$75 = (113)_8$$

Base	Given number	Remainder
16	75	
16	4	11 = B
	0	4

$$75 = (4B)_{16}$$

Sample questions

- **Convert the octal number 157 to its binary equivalent.**
- **Express the hexadecimal number 2C in decimal form.**
- **Convert the decimal number 245 to its octal and hexadecimal equivalents.**
- **Explain the process of converting a binary number 101101 to hexadecimal.**
- **If a number in binary is 111010, what is its value in BCD (Binary-Coded Decimal) code?**

Computer Arithmetic

- **Number System : Binary Numbers**
 - **Fixed Numbers (Unsigned numbers)**
 - **n bit Binary : 0 to $2^n - 1$...total 2^n values**
 - **4 bit binary : 0 to 15 ($2^4 - 1$)...total 16 values**
 - **Signed Numbers**
 - **n bit Binary : 2^n values, half values negative, half positive ,
MSB reserved for indication of Sign**
 - **0 - positive , 1 - negative**
 - **1's Complement**
 - **2's Complement**

Signed Number Representation

1's complement Representation

- Complement of binary number : replace 1 by 0 , and 0 by 1

0100 ---- 1011

- sign bit is a most significant bit
- 0 = positive number
1 = negative number

Signed Number Representation

- 1's complement Representation

Binary Value	Unsigned	Signed 1's complement
000	0	0
001	1	1
010	2	2
011	3	3
100	4	-3
101	5	-2
110	6	-1
111	7	-0

Signed Number Representation

2's complement Representation

- 2's complement = complement of a number + 1
- Add 1 to 1's complement
- $1011 + 1 = 1100$

Example of 2's Complement

5 = 0000101

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓

11111010

+1

-5 = 11111011

Complement Digits

Add 1

0 1 1 0 1 1 1 0 ← Original binary value

1 0 0 1 0 0 0 1 ← 1's complement

1 0 0 1 0 0 0 1
+ 1
1 0 0 1 0 0 1 0

2's complement

-13 = 11110011

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓

00001100

+1

13 = 00001101

Complement Digits

Add 1

Signed Number Representation

- **Find 2's complement Representation and its value**
 - 1000
 - 0100
 - 111001
 - 101010
 - 100000

Signed Number Representation

Binary	Unsigned	Signed 1's complement	Signed 2's complement
000	0	0	0
001	1	1	1
010	2	2	2
011	3	3	3
100	4	-3	-4
101	5	-2	-3
110	6	-1	-2
111	7	-0	-1



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



Signed Number Representation

Binary	Unsigned	1's complement	2's complement	Binary	Unsigned	1's complement	2's complement
0000	0	0	0	1000	8	-7	-8
0001	1	1	1	1001	9	-6	-7
0010	2	2	2	1010	10	-5	-6
0011	3	3	3	1011	11	-4	-5
0100	4	4	4	1100	12	-3	-4
0101	5	5	5	1101	13	-2	-3
0110	6	6	6	1110	14	-1	-2
0111	7	7	7	1111	15	-0	-1

Sample examples

Convert -17 to binary 1's complement and 2's complement representation

17 = 010001

1's complement form ---- 101110

2's complement form ---- 101111

Convert -49 to binary 1's complement and 2's complement representation

49 = 0101111

1's complement form ---- 1010000

2's complement form ---- 1010001

Binary addition

Add 12+ 7 in binary

12 = 01100 , 7 = 0111

	0	1	1	0	0
+	0	0	1	1	1
Cy	1	1			
sum	1	0	0	1	1

10011 = 19

Add 17+ 13 in binary

17 = 10001 , 13 = 1101

	1	0	0	0	1
+	0	1	1	0	1
Cy	0	0	0	1	
sum	1	1	1	1	0

11110 = 30

Binary Subtraction – 1's complement method

Subtract 12- 7 in binary

12 = 01100 , 7 = 0111

Take 1's complement of 7

= 11000 and add to 12

If there is carry add to result

		0	1	1	0	0
+		1	1	0	0	0
Cy		1				
sum	1	0	0	1	0	0
						1
		0	0	1	0	1

00101 = 5 result

Subtract 7-12 in binary

12 = 01100 , 7 = 0111

Take 1's complement of 12

= 10011 and add to 7

If there is NO carry result is negative , take 1's complement

		0	0	1	1	1
+		1	0	0	1	1
Cy		0	1	1	1	
sum	0	1	1	0	1	0

1's complement of 11010 = 00101

hence -5 result

Binary Subtraction – 1's complement method

Subtract 15- 11 in binary

15 = 01111 , 11= 01011

**Take 1's complement of 11
= 10100 and add to 12**

If there is carry add to result

		0	1	1	1	1
+		1	0	1	0	0
Cy		1	1			
sum	1	0	0	0	1	1
						1
		0	0	1	0	0

00100 = 4 result

Subtract 11-15 in binary

15 = 01111 , 11 = 01011

**Take 1's complement of 15
= 10000 and add to 11**

**If there is NO carry result is
negative , take 1's complement**

		0	1	0	1	1
+		1	0	0	0	0
Cy						
sum	0	1	1	0	1	1

1's complement of 11011 = 00100

hence -4 result

Binary Subtraction – 2's complement method

Subtract 12- 7 in binary

12 = 01100 , 7 = 0111

Take 2's complement of 7

= 11000 +1 = 11001

and add to 12

If there is carry neglect, the result is positive

		0	1	1	0	0
+		1	1	0	0	1
Cy		1				
sum	1	0	0	1	0	1
		0	0	1	0	1

00101 = 5 result

Subtract 7-12 in binary

12 = 01100 , 7 = 0111

Take 2's complement of 12

= 10011 +1 = 10100

and add to 7

If there is NO carry result is negative , take 2's complement

		0	0	1	1	1
+		1	0	1	0	0
Cy		0	1			
sum	0	1	1	0	1	1

2's complement of 11011 = 00101

hence -5 result

Binary Subtraction – 2's complement method

Subtract 14- 6 in binary

14 = 01110 , 6= 00110

Take 2's complement of 6

= 11001 +1 = 11010 and add to 14

If there is carry , neglect, the result is positive

		0	1	1	1	0
+		1	1	0	1	0
Cy		1	1	1		
sum	1	0	1	0	0	0

01000 = 8 result

Subtract 7-14 in binary

14 = 01110 , 7 = 00111

Take 2's complement of 14

= 10001+1 = 10010 and add to 7

If there is NO carry result is negative , take 1's complement

		0	0	1	1	1
+		1	0	0	1	0
Cy			1	1		
sum	0	1	1	0	0	1

2's complement of 011001 = 00111

hence -7 result

Gray Code

Gray code is a binary numeral system where each successive value differs by only one bit, which minimizes errors during transitions in digital systems

Decimal	Binary Code	Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100

Generation of Gray Code from Binary

Let $b_3b_2b_1b_0$ is a 4 bit binary number

To generate equivalent Gray code

1. $g_3 = b_3$...retain Most significant bit (MSB)
2. $g_2 = b_3 \text{ xor } b_2$
3. $g_1 = b_2 \text{ xor } b_1$
4. $g_0 = b_1 \text{ xor } b_0$

Example :

Binary	b_3	b_2	b_1	b_0
	1	1	0	0
g_3	1			
g_3g_2	1	0		
$g_3g_2g_1$	1	0	1	
$g_3g_2g_1g_0$	1	0	1	0

Binary	b_3	b_2	b_1	b_0
	0	1	1	0
g_3	0			
g_3g_2	0	1		
$g_3g_2g_1$	0	1	0	
$g_3g_2g_1g_0$	0	1	0	1

BCD Code

Binary-Coded Decimal (BCD) is a numbering system where each decimal digit (0–9) is represented by its own four-bit binary equivalent. (even if number can be represented in lesser digits it is represented in 4 bit)

0 = 0000

1 = 0001

2 = 0010

3 = 0011

4 = 0100

5 = 0101

6 = 0110

7 = 0111

8 = 1000

9 = 1001

BCD Code

Multidigit numbers are represented by combinations of 4 bit representations of each digit

17 = 0001 0111

259 = 0010 0101 1001

64 = 0110 0100

- **Representing numbers in binary form is easy as conversion is not required**
- **The value is for each group of 4 bits and this is not positional number system**

Write following numbers in BCD form

34 = 0011 0100

182 = 0001 1000 0010

401 = 0100 0000 0001

628 = 0110 0010 1000

Parity for Error Detection

- In digital electronics, parity is an error-checking technique that uses an extra binary digit, called a parity bit, to ensure data integrity.
- It is added to a binary code so that the total number of '1's in the data is either always even (even parity) or always odd (odd parity).
- At the receiving end, the parity bit is checked to detect if any single-bit errors have occurred during transmission.

Parity for Error Detection

Parity generator:

- A parity generator adds a parity bit to a block of data.
- **Even parity:** A '0' is added if there is an even number of '1's, and a '1' is added if there is an odd number of '1's, to make the total count **even**.
- **Odd parity:** A '1' is added if there is an even number of '1's, and a '0' is added if there is an odd number of '1's, to make the total count **odd**.

Transmission: The data along with the parity bit is transmitted.

Parity checker:

- The receiver counts the number of '1's in the received data, including the parity bit.
- **Even parity:** If the count is even, the data is accepted as correct. If the count is odd, a parity error is detected.
- **Odd parity:** If the count is odd, the data is accepted. If the count is even, an error is detected.

Error detection:

- If the parity check fails, the data is flagged as corrupted. This method can only detect single-bit errors; if two bits flip, the parity will still appear correct.

Parity for Error Detection

Parity generator:

- Given data : 1101
- **Even parity:** Parity Bit = 1 so that total no of '1's **even**
- **Odd parity:** Parity Bit = 0 so that total no of '1's **odd**.

Transmission: The data along with the parity bit is transmitted.

Parity checker:

- The receiver counts the number of '1's in the received data, including the parity bit.
- **Even parity:** Received Data : 11010 Count is odd there is error
- **Odd parity:** Received Data : 11010Count is odd so data is correct